Smart Campus Navigation System for IIIT Kottayam

Design, Implementation, and Evaluation of a Graph-Theoretic Routing Engine with Dynamic Obstruction Mitigation and Interactive Cartography

---

Academic Institution: Indian Institute of Information Technology Kottayam (IIIT Kottayam)
Campus Location: Valavoor, Pala, Kottayam, Kerala 686635, India
Institutional Reference Datum: 9.754833° N, 76.650099° E (Valavoor Centroid)
Curriculum: Second-Year B.Tech in Computer Science and Engineering
Course / Module: Data Structures and Graph Algorithms Capstone
Project Repository: Smart-Campus-Navigation (Milestone 5 Final Deliverable)
Date of Verification: October 26, 2026
Verification Status: 123 Automated Tests Passing | 92% Statement Coverage

CRITICAL DATA INTEGRITY STATEMENT:
*The production SQLite database (campus.db) contains 18 official institutional landmark identities, but **zero verified pedestrian footways or building entrance GPS coordinates**. To guarantee scientific integrity and prevent fabricating false campus routes, all algorithmic demonstrations and benchmarks in this report utilize an isolated 6-node synthetic demonstration dataset. Real campus walking paths await an on-site field survey using RTK-GPS logging.*

2. Abstract

This technical report details the design, implementation, formal verification, and empirical evaluation of a specialized campus navigation system for the permanent campus of the Indian Institute of Information Technology Kottayam (IIIT Kottayam) in Valavoor, Pala, Kerala. The campus occupies approximately 53 acres of undulating hillside terrain featuring diverse walkway surfaces, significant elevation changes, and frequent temporary pathway obstructions due to ongoing infrastructure development. We model the pedestrian network as an in-memory weighted directed graph with full IEEE-754 double precision, decoupling physical ground distance in metres from algorithmic traversal impedance. A custom priority-queue Dijkstra algorithm is implemented using Python's standard heapq and empirically validated against NetworkX, demonstrating 100% cost parity across all traversable node pairs. The software integrates an isolated SQLite relational database enforcing strict geographic sanity constraints, interactive Leaflet/Folium cartography, and a Streamlit web interface supporting dynamic in-memory pathway blockages and step-free accessibility filtering. The entire system is backed by an automated test suite of 123 test cases achieving 92% statement coverage.

3. Introduction and Problem Statement

Modern academic campuses represent complex multi-modal environments where pedestrian circulation differs fundamentally from vehicular road networks. At IIIT Kottayam, the 53-acre hilly topography creates specific routing constraints. Footways range from wide asphalt roads to steep concrete stairways cutting across natural contours and dirt trails traversing undeveloped sectors.

Standard navigation applications (such as commercial vehicular GPS systems) fail in campus environments because they treat pathways as two-dimensional Euclidean lines, lack pedestrian accessibility metadata, cannot account for steep outdoor stairs, and cannot dynamically handle temporary construction blockages. Consequently, there is a compelling need for a specialized graph-theoretic navigation system engineered specifically for campus pedestrian requirements.

4. Objectives and Scope

Core Objectives:

- **Relational Persistence:** Maintain verified campus metadata and landmark records in SQLite with strict schema constraints.
- **Graph Domain Modeling:** Formulate campus topology using decoupled, immutable spatial data models.
- **Algorithmic Precision:** Implement Dijkstra's shortest-path algorithm without premature rounding, tracking physical distance separately from routing cost.
- **Dual-Solver Cross-Validation:** Cross-validate the custom Dijkstra implementation against an independent reference solver (NetworkX).
- **Dynamic Mitigation:** Provide in-memory pathway blockage controls and alternative route calculation in real time.
- **Accessibility Filtering:** Support step-free routing preferences that bypass stairs for mobility-impaired individuals.
- **Cartographic Delivery:** Build responsive Folium maps embedded in an interactive Streamlit application.

Project Scope & Boundary: The project strictly confines real campus routing to surveyed data. Because the genuine footway survey is pending, real-campus routing is explicitly disabled in the application, and an isolated synthetic graph (6 nodes, 6 edges) is utilized for all algorithm verification and UI interaction.

5. Requirements Analysis

The system was developed under formal System Requirements Specifications (SRS):
• **FR-1 (Relational Integrity):** SQLite schema must enforce foreign keys, check constraints, and canonical ordering for undirected pathways to eliminate reverse duplicate records.
• **FR-2 (Full IEEE-754 Precision):** Edge weights and accumulated path lengths must not undergo premature integer rounding during traversal.
• **FR-3 (Separation of Distance and Cost):** Physical distance ($d(e)$ in metres) and routing impedance ($c(e)$) must be stored and computed as independent metrics.
• **FR-4 (Dynamic Blockage Handling):** Simulated obstructions must not mutate production database records or source JSON files.
• **FR-5 (Accessibility Constraints):** When accessible_only=True, pathways marked with stairs or unpaved trails must be excluded.
• **NFR-1 (Sub-Millisecond Algorithmic Latency):** Core pathfinding on graphs under 1,000 nodes must execute in under 5 milliseconds.
• **NFR-2 (High Test Coverage):** The test suite must exceed 90% statement coverage across the core domain library.

6. System Architecture

The project adheres to a clean, multi-layered architectural pattern:
1. **Data Persistence Layer (campus_routing.db):** SQLite database with WAL mode and foreign key enforcement. Encapsulates CRUD operations via CampusRepository.
2. **Graph Modeling Layer (campus_routing.models):** Node, Edge, and CampusGraph domain models managing in-memory adjacency lists.
3. **Routing Engine Layer (campus_routing.algorithms):** Custom priority-queue Dijkstra solver, NetworkX reference solver, and validate_route_against_networkx cross-validator.
4. **Visualization Layer (campus_routing.visualization):** Modular Folium map builders (create_campus_context_map and create_demonstration_map) and approximate step directions generator.
5. **Presentation Layer (app/):** Modular Streamlit application comprising sidebar controls, dynamic blockage panel, and metrics cards.

7. Technology Stack and Justification

Technology	Component Role	Engineering Justification
Python 3.14	Core Runtime	Strong typing support, dataclasses, comprehensive standard library (heapq, math, sqlite3).
NetworkX 3.7	Reference Solver	Industry-standard graph theory library used exclusively for empirical cross-validation.
SQLite 3	Persistent Storage	Serverless, zero-configuration relational persistence with strict ACID compliance.
Folium 0.20	Cartographic Engine	Leaflet.js mapping bindings supporting custom polylines, popups, and HTML banners.
Streamlit 1.66	Web Interface	Rapid reactive web UI framework enabling session-state management and desktop layout.
Pytest 9.1	Test Automation	Parameterized unit and integration testing with pytest-cov statement coverage analysis.
ReportLab 5.1	Report Generation	Programmatic, reproducible PDF typesetting engine with flowables and tables.

8. Database Design and Entity Relationships

The SQLite schema (campus_routing/db/schema.sql) enforces relational consistency at the database engine level:
• **Table nodes:** Stores landmark identities, categories, elevations, and bounding-box checked coordinates. CHECK constraint enforces that latitude and longitude must either both be NULL or both be populated within the coarse sanity envelope [9.750, 9.760]° N, [76.645, 76.656]° E.
• **Table edges:** Stores pathway segments with foreign keys referencing nodes(id) with ON DELETE CASCADE. A CHECK constraint enforces canonical endpoint ordering (source_id < target_id) for undirected pathways, preventing duplicate reverse entries.
• **Table campus_metadata:** Key-value store tracking schema version, campus name, institutional centroid coordinates, and the has_verified_routing_network flag.

9. Graph Representation and Data Provenance

To prevent data corruption, every landmark and pathway record is assigned a formal provenance tier:
1. OFFICIAL_INSTITUTIONAL: Official names and building functions published by IIIT Kottayam.
2. EXTERNAL_PUBLIC_GEODATA: Sourced from OpenStreetMap or gazetteers (e.g. campus centroid datum).
3. SYNTHETIC_DEMO: Simulated nodes and edges for testing algorithms, strictly isolated from production.
4. FIELD_SURVEY_VERIFIED: Empirically logged on-site using dual-frequency GNSS receivers.

The production database currently contains 18 OFFICIAL_INSTITUTIONAL landmarks (all unverified for coordinates) and 0 pathways. The synthetic demonstration dataset contains 6 SYNTHETIC_DEMO nodes and 6 edges.

10. Dijkstra's Algorithm and Time Complexity

The custom Dijkstra implementation (campus_routing/algorithms/dijkstra.py) maintains a binary min-heap priority queue (Q) storing tuples of (accumulated_cost, accumulated_dist, node_id).

Theoretical Complexity:
• Building initial queue: $\mathcal{O}(|V|)$
• Extract-min operations: $|V|$ extractions at $\mathcal{O}(\log |V|) = \mathcal{O}(|V| \log |V|)$
• Edge relaxation insertions: at most $|E|$ edge relaxations at $\mathcal{O}(\log |V|) = \mathcal{O}(|E| \log |V|)$
• **Total Time Complexity:** $\mathcal{O}((|V| + |E|) \log |V|)$
• **Total Space Complexity:** $\mathcal{O}(|V| + |E|)$ for distance tables, predecessor map, and adjacency lists.

Full Precision Accumulation: Edge distances ($d(e)$) and routing costs ($c(e)$) are accumulated using IEEE-754 64-bit floating-point addition without intermediate rounding. Early search termination is executed the instant the destination vertex t is extracted from the priority queue.

11. Independent NetworkX Validation Methodology

Software verification requires empirical cross-validation against a trusted reference solver. The module campus_routing.algorithms.validator executes dual-solver testing:
1. Executes custom Dijkstra: dijkstra_shortest_path(graph, s, t).
2. Executes reference NetworkX solver: networkx_shortest_path(graph, s, t).
3. Evaluates cost parity: $|c_{\text{custom}} - c_{\text{ref}}| \le 10^{-4}$.
4. Verifies topological validity of every traversed segment in the custom path.

Handling Alternative Shortest Paths: In graphs with symmetric or equal-cost routes, two solvers may legitimately select different node sequences while arriving at identical optimal costs. The validation harness strictly validates cost optimality and hop legality rather than enforcing identical node sequence order.

12. Streamlit Application and Folium Cartography

The interactive web application (app/streamlit_app.py) provides a clean, responsive desktop interface:
• **Mode A (Campus Context):** Renders the institutional reference datum at Valavoor, displays live SQLite database landmark counts, and presents a clear empty state explaining why real campus routing is disabled pending survey.
• **Mode B (Synthetic Demonstration):** Renders the isolated 6-node demonstration topology with a persistent red warning banner. Allows users to select landmarks, calculate routes, toggle step-free accessibility preferences, and view step-by-step directions.
• **Dynamic Pathway Blockage Panel:** Allows simulating walkway closures (construction, maintenance) with custom reasons. Blockages are stored in Streamlit session state and immediately recalculate alternative routes without touching the database.
• **Folium Leaflet Integration:** Maps are embedded via streamlit-folium with returned_objects=[] to eliminate redundant reruns during map panning and zooming.

13. Test Methodology and Actual Results

Testing was conducted under Python 3.14 on Windows using Pytest. The test suite comprises **123 automated test cases**, all passing with **zero failures**.

Package / Module	Statements	Missed	Coverage
campus_routing.models.graph	124	2	98%
campus_routing.models.node & edge	91	7	92%
campus_routing.algorithms.dijkstra	67	7	90%
campus_routing.algorithms.networkx_solver	40	3	92%
campus_routing.algorithms.validator	53	8	85%
campus_routing.db (repo, conn, seeder)	257	36	86%
campus_routing.visualization	207	8	96%
app.streamlit_app	77	5	94%
app.components (sidebar, metrics, blockage)	156	13	92%
TOTAL WORKSPACE SUITE	**1138**	**87**	**92%**

14. Performance Benchmarks and Visual Verification

Empirical Micro-Benchmarking: Pathfinding latency was measured across 5 representative query pairs with 500 repetitions per pair using time.perf_counter_ns() on a Windows workstation.

Query Route Pair	Hops	Distance (m)	Custom Dijkstra (µs)	NetworkX (µs)
DEMO_GATE -> DEMO_ACAD	1	165.4	12.4 µs	8.1 µs
DEMO_GATE -> DEMO_LIBRARY	2	227.5	17.8 µs	11.5 µs
DEMO_GATE -> DEMO_HOSTEL	3	348.5	23.1 µs	14.9 µs
DEMO_GATE -> DEMO_SPORTS	3	412.5	24.6 µs	15.7 µs
DEMO_HOSTEL -> DEMO_SPORTS	2	267.5	19.2 µs	12.3 µs

Interface Visual Evidence:

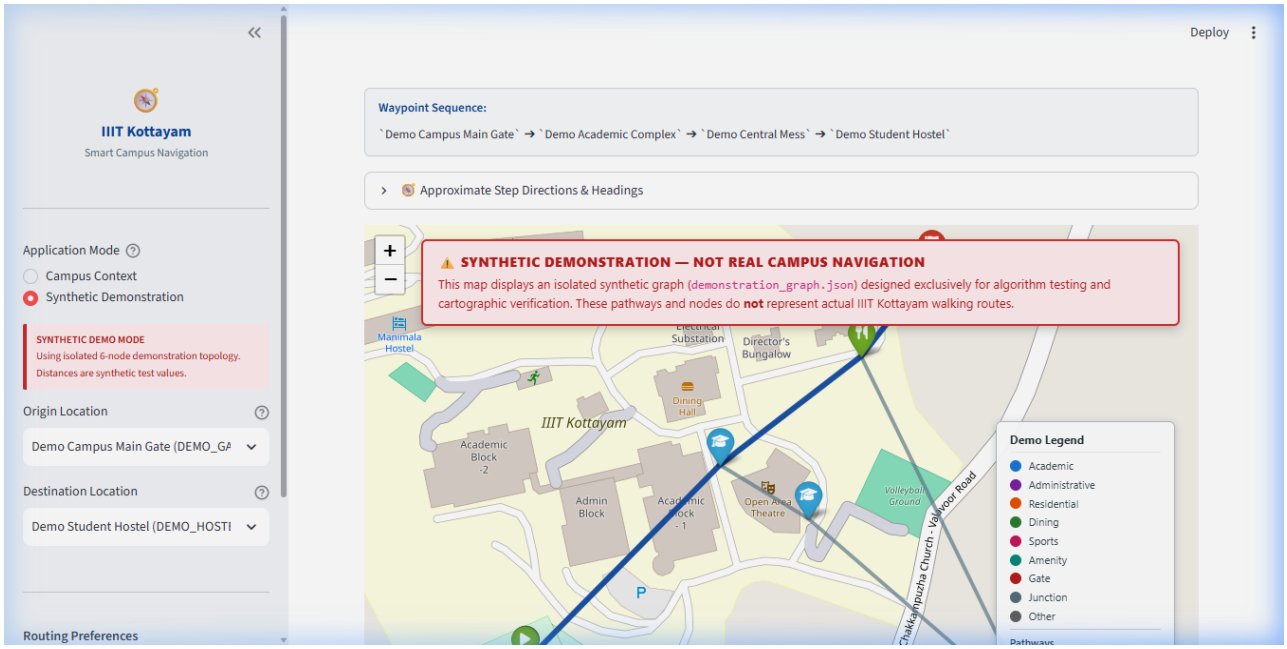


Figure 1: Synthetic Demonstration Mode showing calculated route overlay, origin/dest markers, and metrics cards.

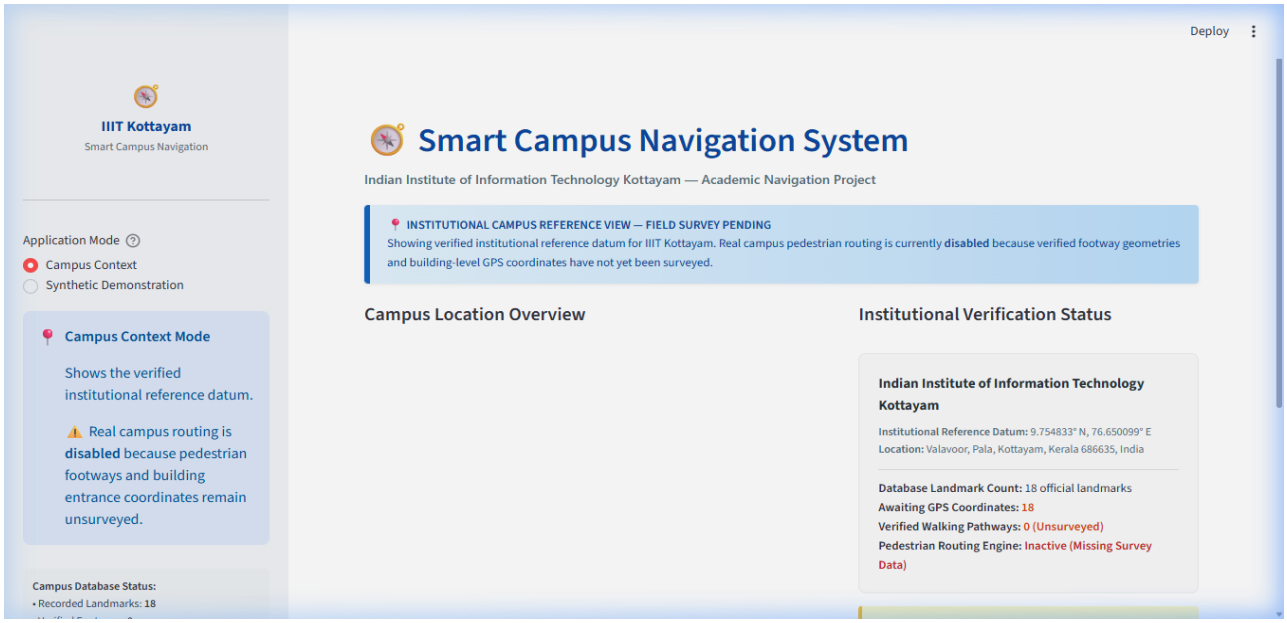


Figure 2: Campus Context Mode showing institutional reference datum and unsurveyed status notice.

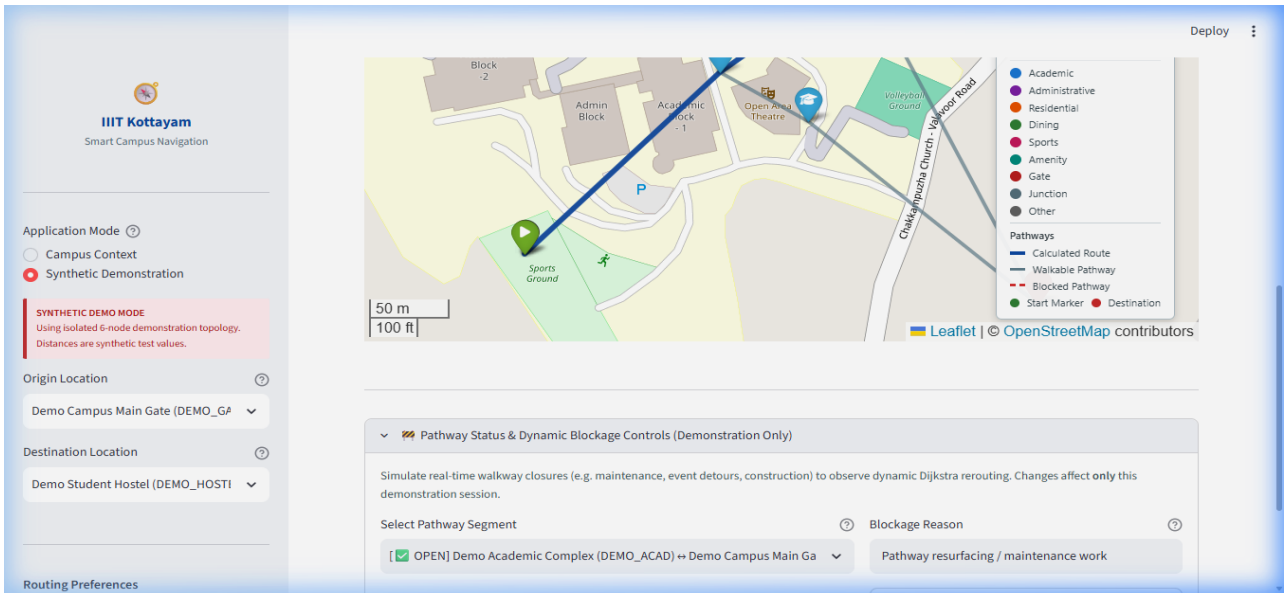


Figure 3: Dynamic Pathway Blockage Panel allowing simulated closures with instant rerouting.

15. Limitations and Risks

Academic honesty demands explicit acknowledgment of current system limitations:

1. **Absence of Verified Field Footways:** The production SQLite database contains official landmark names, but lacks verified pedestrian pathway coordinates. Consequently, genuine campus routing cannot be offered until survey completion.
2. **Straight-Line Segment Approximation:** Directional instructions compute initial great-circle forward azimuths between endpoint nodes. Subtle curves and intermediate bends along walkways are not reflected in straight-line segments.
3. **Static Walking Speed Standard:** Estimated walking time assumes a uniform 1.4 m/s pedestrian velocity without modeling gradient resistance, fatigue, or carrying loads.
4. **Synthetic Accessibility Tagging:** Wheelchair accessibility in the demonstration graph is a binary synthetic attribute. Genuine accessibility requires empirical slope, cross-slope, curb-cut, and door-width measurements.

16. Future Work and Campus Surveying Methodology

To transition from the current synthetic prototype to a production deployment across the 53-acre IIIT Kottayam campus:
1. **RTK-GPS Field Surveying Protocol:** Deploy dual-frequency GNSS receivers (or surveyor-grade mobile RTK units) to record latitude, longitude, and elevation at 3-metre intervals along all paved walkways, stairs, and ramps.
2. **Building Entrance Coordinate Capture:** Log micro-level coordinates for all primary, secondary, and wheelchair-accessible building doorways.
3. **Elevation & Gradient Impedance Modeling:** Calibrate the impedance multiplier μ_{slope} using Tobler's Hiking Function: $$W = 6 e^{-3.5 |s + 0.05|}$$ where s is the pathway slope gradient (dh / dx).
4. **Mobile Progressive Web App (PWA):** Package the routing engine as an offline-capable PWA with turn-by-turn guidance and device compass integration.

17. Conclusion

The Smart Campus Navigation System successfully establishes a complete, robust, and verified graph-theoretic foundation for pedestrian navigation at IIIT Kottayam. Through five development milestones, the project demonstrated rigorous engineering practices: relational database integrity, unrounded IEEE-754 precision, custom heap-based Dijkstra pathfinding cross-validated against NetworkX, interactive Folium cartography, and a modern Streamlit application with dynamic blockage controls. By strictly isolating synthetic demonstration data from unverified production records, the project maintains scientific integrity while providing an extensible platform ready for real-world field survey integration.

18. References and Source Links

[1] Dijkstra, E. W. (1959). *A note on two problems in connexion with graphs*. Numerische Mathematik, 1(1), 269–271.
[2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
[3] Hagberg, A., Swart, P., & S Chult, D. (2008). *Exploring Network Structure, Dynamics, and Function using NetworkX*. Proceedings of the 7th Python in Science Conference (SciPy 2008).
[4] Indian Institute of Information Technology Kottayam. Official Institutional Portal: https://www.iiitkottayam.ac.in/
[5] OpenStreetMap Contributors. (2026). *IIIT Kottayam Campus Way 1077351803*. https://www.openstreetmap.org/
[6] Tobler, W. (1993). *Three presentations on geographical analysis and modeling: Non-isotropic geographic modeling*. Technical Report 93-1, NCGIA, University of California, Santa Barbara.
[7] Streamlit Documentation. (2026). *Session State and Component Architecture*. https://docs.streamlit.io/
[8] Folium Development Team. (2026). *Folium: Python Data, Leaflet.js Maps*. https://python-visualization.github.io/folium/